

paged

A reference document for the paged engine — sixteen pages, set and rendered by the software they describe.

Contents

- 01 Cover
- 02 Contents
- 03 Editorial — how a page is made
- 04 Editorial, continued
- 05 Raster images
- 06 Vector drawing
- 07 Spreadsheets
- 08 Web content
- 09 Database publishing
- 10 Word documents
- 11 Tables
- 12 Layers
- 13 Conditional text
- 14 Master pages
- 15 Colour and transparency
- 16 Colophon

The feature registry behind every claim in this document lives at docs.paged.media.

How a page is made

Every page in this document was produced the way the engine produces any page. An IDML package is parsed into a document model; the model resolves into a scene; the scene's text is composed line by line into positioned glyph runs; the composed page is flattened into a display list; and the display list is rasterised. Five stages, one direction, and no state passed sideways between them.

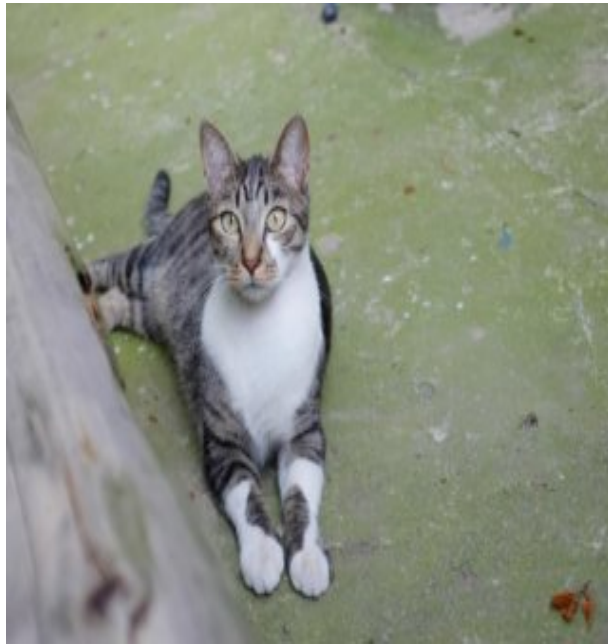
The parser is the part that has to be humble. InDesign's format is two decades of accumulated decisions, and a renderer that handles only the tidy half of it is a demo rather than a tool. So the parser reads what is on the page instead of what it wishes were there: attributes it does not model survive a round trip untouched, an unfamiliar DOMVersion is not fatal, and every element it cannot interpret is carried through to export unchanged. Fidelity is **measured, not asserted** — the gate renders each page and compares it against InDesign's own PDF with a colour-difference metric and a structural-similarity score, so a regression arrives as a number rather than as an opinion.

Text is where a layout engine earns its keep. Lines are broken by the Knuth–Plass algorithm over a whole paragraph rather than greedily, one line at a time, which is why justified columns look settled instead of blotchy; hyphenation, tracking, kerning and the word- and letter-space limits all feed the same badness calculation.

A
file
opened
by
some
body
who
has
none
of
the
plu

Raster — a photograph, placed and corrected

One 320 × 240 progressive JPEG, placed twice. The left frame is the engine's own placed graphic; the right one went through `paged.image`, which decoded it, read a histogram off the pixels and corrected it.



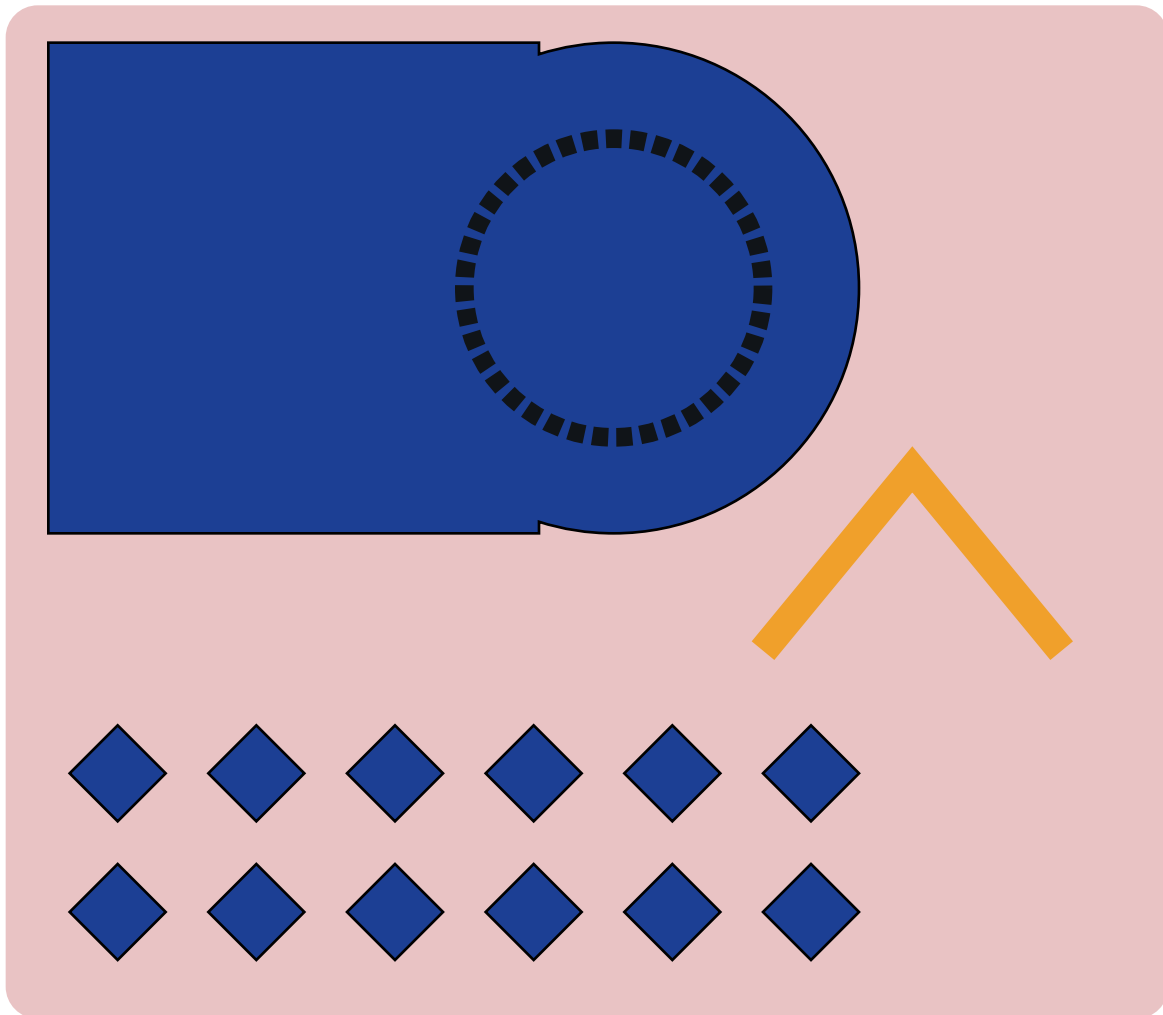
Placed. A link, a `FillProportionally` fitting and level-0 tiles through the resource channel — what a placed graphic is in an IDML package, with no plugin involved.

Corrected. The same file through the raster importer, then Auto-enhance: a percentile auto-levels black and white point and a gray-world white balance, both derived in Rust from this image's own histogram. These pixels come from the plugin's tile provider, which fits the image to the frame proportionally rather than by the frame's crops — which is why the two frames, given identical fittings, are framed differently.

The correction lives in the plugin's session and in the tiles it serves the renderer; the link and the fitting live in the document. Reopening the file gives back both frames and their links, and the correction returns when `paged.image` re-ingests. Photograph: the `image-rs` decoder test suite, MIT OR Apache-2.0, recorded in the assets README.

Vector — an SVG, lowered to native paths

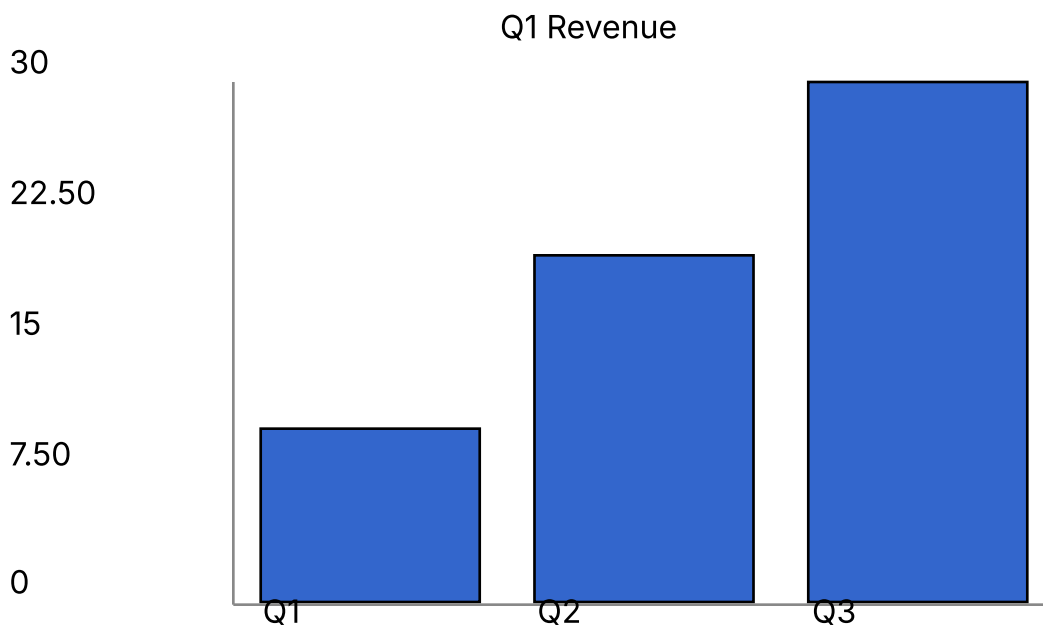
One SVG file through the host importer registry. `paged.draw` lowered every contour with the same `insertPath` the pen tool commits, then four of its ninety-two commands edited the result in place.



The square and the disc were merged by a pathfinder union; the ring's stroke carries a dash preset; the plate behind the mark has live rounded corners; and the diamond field is one motif repeated on a grid, rebuilt as real artwork rather than tracked as a live link. `paged.draw` is the one plugin in this document with nothing to rehydrate on reopen: there is no plugin frame here, only anchors, swatches and strokes.

Spreadsheet — a workbook, lowered twice

One .xlsx opened through the host file picker. paged.sheet parsed it in Rust, then put it on the page twice: once as vector artwork from its chart engine, once as a native table from a cell range.



Region	Revenue
Q1	10
Q2	20
Q3	30

The bars are not a picture of a chart. The chart subsystem ran to a frozen geometry description and the bundle wrote that out as native paths and text frames, so the plot exports to IDML and to PDF as ordinary artwork and a reader without the plugin still sees it. The table below it is a native IDML table, and its column widths were measured through this document's own font metrics rather than guessed from character counts — narrowly enough that the second column's header wraps by a hair, which is left as it was measured. The workbook itself travels in the container as a plugin part, so reopening the file gives the numbers back, live.

HTML and CSS, placed on the page

A web frame is a rectangle carrying a source envelope in document metadata. The plugin lays the source out with a real browser engine, submits the paint as an in-frame scene layer, and then bakes that paint down to native swatches, rectangles and text frames so the result exports like anything else on this page.

Spring line sheet

Authored as HTML and CSS. Laid out by the engine, baked to native page items.

Style	Colourway	Retail
Aster coat	Ink	420
Bellwether shirt	Chalk	145
Corbel trouser	Slate	190
Dovetail knit	Ochre	165

Rendered by the web engine and baked to 41 native page items. The bake is what carries this block into showcase.idml and into a PDF: after it, nothing here needs the plugin to draw.

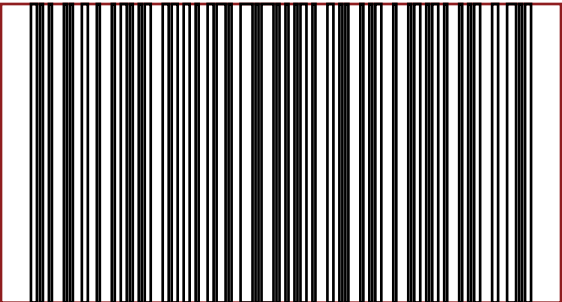
Publishing from a database

A CSV registers through the plugin's DuckDB query engine, its columns become bindings with engine-authored expressions, and the resolved values lower into the document as native content. The field below is live: it re-resolves against the data rather than being pasted in as text.

Bound variable — the record's name

Ada Lovelace

Bound barcode — Code-128, as vector modules



Bound table — one row per record

Column 1
#PARSE
#PARSE
#PARSE

The name above is a tagged placeholder field, not pasted text: the engine resolved it from the CSV through DuckDB and a data refresh re-resolves it in place. The barcode is 46 native filled paths scaled to its frame — real page geometry, so it stays sharp at any output resolution and needs no asset store. The table below it is a native Paged table, one row per record.

A Word document, poured in

The memo below started as a .docx. The plugin lowered its OOXML to native paragraphs and character runs and created the paragraph and character styles it needs before applying them, so what you are reading is Paged content — not an embedded viewer. The original file travels inside the container beside it.

Plain body text.

A Centered Heading

Mix of normal and bold red text.

Poured onto this page as a native story of 66 characters, with the paragraph and character styles the .docx declared synthesized into the document's own catalog. The source file travels beside it inside the container.

Part Two · Plates and Notes

Table 1 — the render pipeline, and what each stage hands to the next.

Stage	Crate	Reads	Writes
Parse	paged-parse	IDML package	Doc u ment model
Resolve	paged-scene	Doc u ment model	Scene graph
Compose	paged-compose	Scene graph	Display list
Rasterise	paged-gpu	Display list	Pixels
One direction, five stages — the same path serves a live canvas, a snapshot in a test, and a PDF bound for a press.			

Layers



Both plates were drawn onto Content. Moving the accent plate to Background inverted which one occludes the other — the renderer sorts by layer before it consults the z table, so this is a different gesture from Bring to Front, and until protocol 62 the engine had no way to express it at all.

Conditional text

A conditional document is one document that ships as several. This sentence is tagged Draft, and exists only in the internal cut. Because a hidden condition is dropped BEFORE composition, the surrounding text reflows to close the space — it does not leave a hole where the run used to be. That is the difference between conditional text and simply colouring something white.

The middle sentence carries the "Draft" condition. Conditions are DEFINED in the document, not authored over the wire: `setConditionVisible` and `activateConditionSet` toggle what already exists, and no mutation mints a condition. This page's definitions come from the generated base fixture.

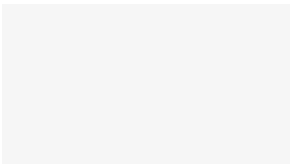
Masters and sections

A master spread stamps running heads, folios and rules onto every page that applies it, and a page can override one stamped item locally without detaching from the master — the override suppresses just that item, and everything else keeps tracking the master. This page also opens a section: numbering restarts at 1 with the prefix B-, so the folio the master carries resolves differently here than on the page before it. The marker is not text the page stores; it is resolved at composition time against the section that contains it.

Colour and transparency



Process · CMYK 78/34/0/6



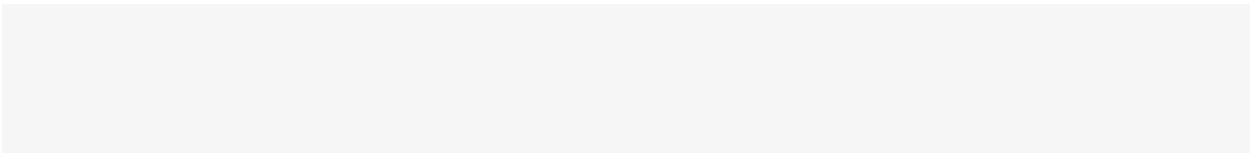
Spot · a named separation



Tint · 35% of the spot



Process · RGB 58/74/92



Gradient · Showcase Blue to Ember, axis at 12°



Blend modes · Normal · Multiply · Screen · Overlay · Darken · Lighten



Opacity · 100% · 80% · 60% · 40% · 20%, the last also at a 40% fill tint

Colophon

This document was not written. It was built — every page authored through the same editor, the same mutation wire and the same plugin bundles a user drives, then saved as a .paged container.

Engine protocol: unreported.

Render backend: CPU (tiny-skia fallback).

Plugin bundles loaded: media.paged.data inferred, media.paged.doc inferred, media.paged.draw inferred, media.paged.image inferred, media.paged.sheet inferred, media.paged.web inferred.

Assets. Type is set in OFL and Apache-2.0 faces from the project's own font set. Raster and vector artwork is MIT/Apache-2.0 or first-party. Nothing here is drawn from the licensed vendor corpus, which grants use but not redistribution — so this file can be shown to anyone.

Not shown, and why. paged.slide is a reserved repository with no commits. paged.pdf opens a PDF by REPLACING the open document, so it cannot contribute a page to a document being built; it and paged.publish are exercised as exit paths instead — this document exports to IDML through the publish adapter and to PDF through the engine's own writer.